

Siddhant Kumar Upmanyu

Senior Systems Architect · Distributed Systems & Platform

Bengaluru, India · [sku20.dev](#) · [github.com/sku0x20](#) · [linkedin.com/in/sku20](#) · [siddhantkumarupmanyu@gmail.com](#) · [Zenodo Preprint](#)

SUMMARY

Senior Systems Architect with architectural responsibility across Google Cloud Platform administration. Specialist in Linux kernel networking, high-throughput databases, JVM runtime optimization, and autonomous SRE triage.

TECHNICAL COMPETENCIES

LANGUAGES	Kotlin, Java, Go, Rust, Zig, C, Shell scripting (POSIX/Bash), SQL
KERNEL & NETWORKING	Linux kernel (<code>nftables</code> , <code>iptables</code>), in-kernel NAT64 (<code>Jool</code>), TCP/UDP socket programming, mTLS & custom PKI (CA/CRL), packet reflection, conntrack tuning
DISTRIBUTED & STORAGE	ClickHouse (columnar modeling, Delta+ZSTD codecs), Redis, PostgreSQL, MongoDB, gRPC, Protocol Buffers, REST
CLOUD & SRE	Google Cloud (IAM, VPC, Cloud Run, Cloud SQL, GCS), AWS (Lambda, EKS spike), Terraform / OpenTofu IaC, Docker, Proxmox VE, GitHub Actions, Linux administration (Debian, Ubuntu)
OBSERVABILITY	Grafana, Loki, VictoriaMetrics, VictoriaLogs, Alloy, <code>async-profiler</code> , JMC, VisualVM, Linux performance tools (<code>sar</code> , <code>vmstat</code> , <code>iostat</code> , <code>pidstat</code>)
METHODOLOGY	Extreme Programming (XP), London-Style TDD, Consumer-Driven Contract Testing (Pact spike), Testcontainers

PROFESSIONAL EXPERIENCE

eGlu Smart Homes (WiZNSystems)

Bengaluru, India · 2022 – Present

Senior Systems Architect (Apr 2026 – Present) | Senior Software Engineer (2024 – 2026) | Software Engineer (2022 – 2024)

Systems Architecture & Kernel Networking

- In-Kernel Dual-Stack NAT64 (`Jool` + `nftables`):** Resolved an edge network partition where cellular IoT hubs operated strictly over IPv6 while backend services were IPv4. Bypassed userspace proxies to preserve CPU credits; deployed the `Jool` kernel module for stateful NAT64 alongside `nftables` DNAT to cleanly partition source port ranges on a shared public IPv4.
- Prerouting Packet Reflection & Dynamic Abuse Mitigation:** Offloaded UDP NAT traversal echo responses directly into the Linux kernel using `nftables` packet reflection (`notrack`) at prerouting priority, avoiding userspace context switches. Implemented kernel-level DDoS mitigation via `nftables` dynamic sets (`limit rate over 10/second burst 20 packets` at prerouting priority -301) dropping sweeps before socket allocation.
- Zero-Downtime Kernel Switching:** Designed zero-downtime deployment pipelines using `iptables` DNAT cutovers across both `PREROUTING` and `OUTPUT` chains, explicitly flushing the `conntrack` state table to switch backend traffic without dropping active connections. Hardened deployment and recovery tooling with atomic routing mutations and pre-flight health checks following high-pressure live incident triage.
- Academic Preprint Publication (Zenodo):** Authored and published *Connection-Agnostic Presence Tracking for Stateless Distributed Backends*: formulated a Redis sorted-set architecture with expiration deadlines and throttled batch writes to track IoT device state with mathematically bounded detection latency.

High-Throughput Telemetry & Storage Systems

- ClickHouse Ingestion Engine (95%+ Compression):** Resolved filesystem inode exhaustion caused by legacy directory log dumps. Designed high-throughput columnar ClickHouse schemas (`FixedString` , `LowCardinality` , Delta + ZSTD codecs, sparse composite primary keys), slashing telemetry footprint by 95%+ (compressing `device_health` telemetry from 17 GB down to 300 MB on disk). Executed zero-downtime partition-level backfilling and restore operations without slow `INSERT INTO` batches, and devised partition-level disaster recovery backup and restore policies.
- Decoupled Ingestion Microservice:** Engineered an auxiliary Go microservice communicating via binary gRPC with strict Protocol Buffers contracts to encapsulate ClickHouse, shielding the primary Kotlin/Spring Boot monolith from database driver coupling.
- Zero-Downtime Storage Architecture:** Architected and orchestrated an end-to-end multi-phase data migration decoupling binary Base64 image blobs from primary MongoDB collections to GCS with zero downtime; designed dual-write API contracts with document guardrails (`imageMigrated`), executed an asynchronous backfill pipeline with immutable GCS caching headers, coordinated backward-compatible mobile rollouts, and purged legacy blobs via `$unset` , slashing database storage by 54x (3.6 GB to 67 MB) and runtime memory by 89% (4.6 GB to 500 MB).
- Production MongoDB DBA & Index Engineering:** Acted as hands-on DBA across heavy-load production MongoDB clusters. Audited slow query logs and execution plans (`explain`), pruned redundant/overlapping indexes to reclaim WiredTiger cache memory and reduce write amplification, refactored hot document schemas, and engineered selective compound indexes (strict Equality-Sort-Range ordering) to eliminate collection scans and in-memory sorts under sustained IoT write traffic.

Concurrency, Runtime & JVM Performance

- **Dynamic IoT-Aware ThreadPool:** Overhauled JVM thread management under bursty IoT packet waves by replacing default thread pools with an adaptive dynamic ThreadPool that scales worker threads based on real-time packet velocity and queue depth.
- **Deep JVM Profiling & Zero-Allocation Hot Paths:** Instrumented live production systems with `async-profiler` and JMC flame graphs. Mitigated allocation churn with buffer pooling and `ThreadLocal` storage; refactored critical packet decoding loops from `String.format` to Java 17 `HexFormat` and bitwise arithmetic, eliminating hot-path CPU bottlenecks. Pinned heap boundaries (`-Xms = -Xmx` at 4GB) and tuned GC for low pause times.
- **Redis Connection Pool Forensics:** Diagnosed and eliminated silent "ghost connections" and thread pool poisoning in Redis Pub/Sub drivers (Lettuce vs. Jedis) that caused cascading application timeouts under transient network partitions.

Platform Engineering, Governance & Cloud FinOps

- **Google Cloud Platform Administration & FinOps:** Google Cloud Platform Administrator managing IAM policies, VPC networks, Cloud Storage, and compute infrastructure. Evaluated spend-based vs. resource-based CUD models and executed a 3-year resource-based Committed Use Discount (CUD) on core compute instances.
- **Observability & Telemetry Infrastructure:** Replaced ad-hoc raw log dumping on production VM disks with centralized log aggregation using Grafana and Loki. Built a pre-production Docker Compose telemetry stack integrating Grafana, Loki, VictoriaMetrics, VictoriaLogs, and Alloy with host and container performance dashboards.
- **Zero-Trust Diagnostic Tooling (`hlogger`):** Built a secure client-server log inspection utility in Go using Mutual TLS (mTLS) with a custom PKI hierarchy (Root CA, Intermediate CA, client certificates, CRL), eliminating unauthenticated engineer SSH access to production instances.
- **On-Prem Staging Parity & Bare-Metal Homelab:** Built an on-prem staging environment replicating production network topology, establishing a mandatory pre-production validation gate. Configured a Proxmox VE bare-metal host provisioned via OpenTofu and `cloud-init` to evaluate container orchestration (k3s, HashiCorp Nomad, AWS EKS spikes).
- **Declarative Marketing Infrastructure:** Decoupled the public marketing website completely from core backend infrastructure into its own isolated GCP project, codifying 100% of Cloud Run, Cloud SQL (PostgreSQL), networking, and IAM via Terraform / OpenTofu.
- **Platform Automation & Integrations:** Extended scene/rule automation engines for multi-hub environments (fragment handling, cross-hub synchronization); built cloud-to-cloud OAuth integrations for Yale smart locks, Google Home, and Alexa; rewrote hardware device provisioning flows and replacement operations for failed nodes.
- **Technical Hiring:** Formulated structured, multi-step technical hiring assessments and evaluation rubrics for backend engineering; interviewed candidates and hired software engineering talent for the platform.

OPEN-SOURCE SYSTEMS & RESEARCH

stopgap Kotlin · Maven Central (`2.8.0`)

Microservice framework on Helidon SE (Nima) + Project Loom virtual threads. Custom compile-time DI via KSP (zero runtime reflection). Three-tier testing harness: unit → in-process integration server → Docker E2E via Testcontainers.

assertgo Go · Generics

Type-safe testing assertion library built with modern Go generics. Provides a fluent assertion API, chainable `Not()`, custom matchers, and zero external dependencies.

relay Zig · Systems

Low-level TCP server in pure Zig with a test-driven approach: raw POSIX socket descriptors, manual memory management without libc runtime dependencies, port binding (`SO_REUSEADDR`), and `SIGPIPE` suppression.

hrh Rust · CLI

Helm Release Helper CLI — automated declarative Helm deployments with diff previews and atomic rollback guarantees via `cargo install`.

avoid Shell / Linux · Distro

Minimal, bootable Linux distribution based on Void Linux engineered for server disaster recovery and lightweight headless appliances.

EDUCATION

Bachelor of Computer Applications (BCA)

2019 – 2022